SRI KRISHNA COLLEGE OF TECHNOLOGY

(An Autonomous Institution)

(Affiliated to Anna University, Chennai and Approved by AICTE)

Accredited by NAAC with 'A' Grade

COIMBATORE – 641 042

DEPARTMENT OF CSE

COURSE CODE	**:**	**23CSC04**
COURSE TITLE	**:**	**PRINCIPLES OF COMPILER DESIGN**
CREDITS/CATEGORY	**:**	**4/PCC**
EXT/INT	**:**	**50/50**
L/T/P	**:**	**3/0/2**

MODULE 1 LEXICAL AND SYNTAX ANALYSIS

Phases of a compiler – Lexical Analysis: Role of Lexical Analyzer – Input Buffering – Specification of Tokens – Recognition of Tokens. Finite Automata – Regular expression to Automata – Minimization of DFA. Syntax Analysis: Role of the parser – Context-Free Grammars – Top-Down parsing: Recursive Descent Parsing – Predictive Parsing. Bottom-up parsing: Shift Reduce Parsing – LR Parsers – Error Handling and Recovery in Syntax Analyzer– LEX and YACC tools.

Upon completion of the course, students shall have ability to		
C202.1	Develop a program to scan source code and identify tokens such as keywords, identifiers, literals, and symbols.	[AP]
C202.2	Implement parsing techniques such as LL(1) and LR parsing to generate parse trees.	[AP]

MODULE 1

Session 1	Introduction to Compilers: Phases of the Compiler
Session 2	Lexical Analysis: Role of Lexical Analyzer
Session 3	Input Buffering – Specification of Tokens – Recognition of Tokens
Session 4	Finite Automata – Regular expression to Automata – Minimization of DFA
Session 5	Syntax Analysis: Role of the parser – Context-Free Grammars
Session 6	Top-Down parsing: Recursive Descent Parsing – Predictive Parsing
Session 7	Bottom-up parsing: Shift Reduce Parsing – LR Parsers
Session 8	Error Handling and Recovery in Syntax Analyzer
Session 9	LEX and YACC tools

Input Buffering in Compiler Design

- The lexical analyzer scans the input from left to right one character at a time. It uses two pointers begin ptr(**bp**) and forward ptr(**fp**) to keep track of the pointer of the input scanned.
- Input buffering is an important concept in compiler design that refers to the way in which the compiler reads input from the source code. In many cases, the compiler reads input one character at a time, which can be a slow and inefficient process.
- **Input buffering is a technique that allows the compiler to read input in larger chunks, which can improve performance and reduce overhead.**

Input Buffering in Compiler Design

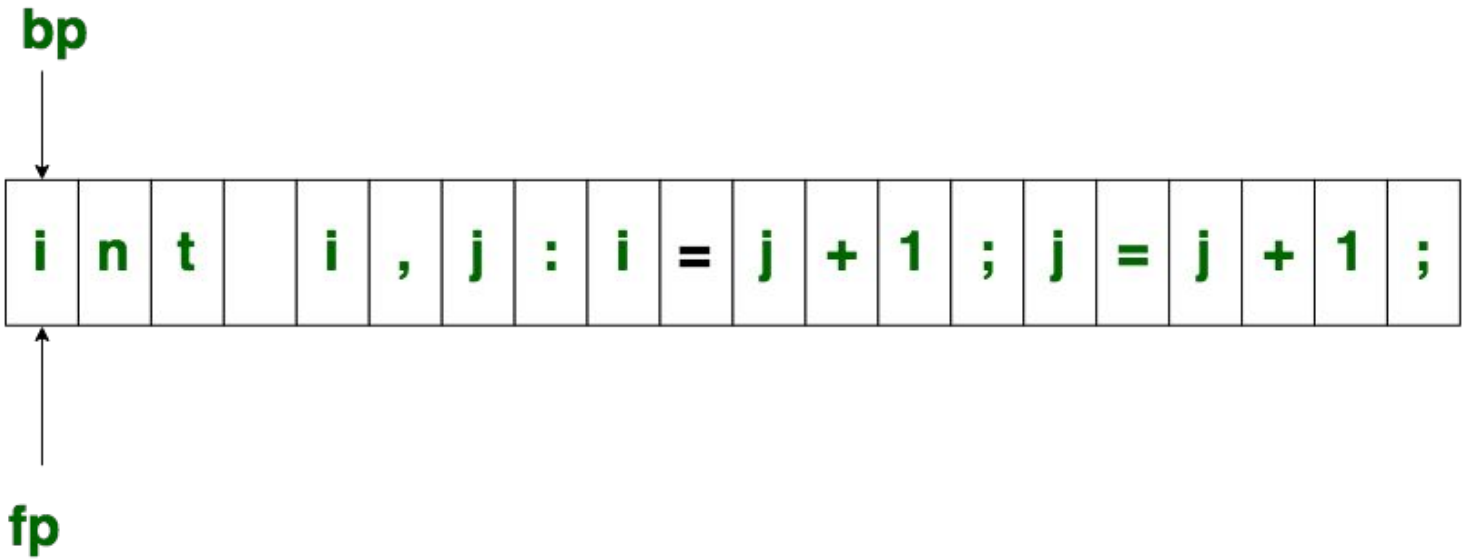
- The basic idea behind input buffering is to read a block of input from the source code into a buffer, and then process that buffer before reading the next block. The size of the buffer can vary depending on the specific needs of the compiler and the characteristics of the source code being compiled.
- For example, a compiler for a high-level programming language may use a larger buffer than a compiler for a low-level language, since high-level languages tend to have longer lines of code.
- One of the main advantages of input buffering is that it **can reduce the number of system calls required to read input from the source code.** Since each system call carries some overhead, reducing the number of calls can improve performance.
- Additionally, input buffering can simplify the design of the compiler by reducing the amount of code required to manage input.

Input Buffering in Compiler Design

- Overall, input buffering is an important technique in compiler design that can help improve performance and reduce overhead. However, it must be used carefully and appropriately to avoid potential problems.

Input Buffering in Compiler Design

- Initially both the pointers point to the first character of the input string as shown below

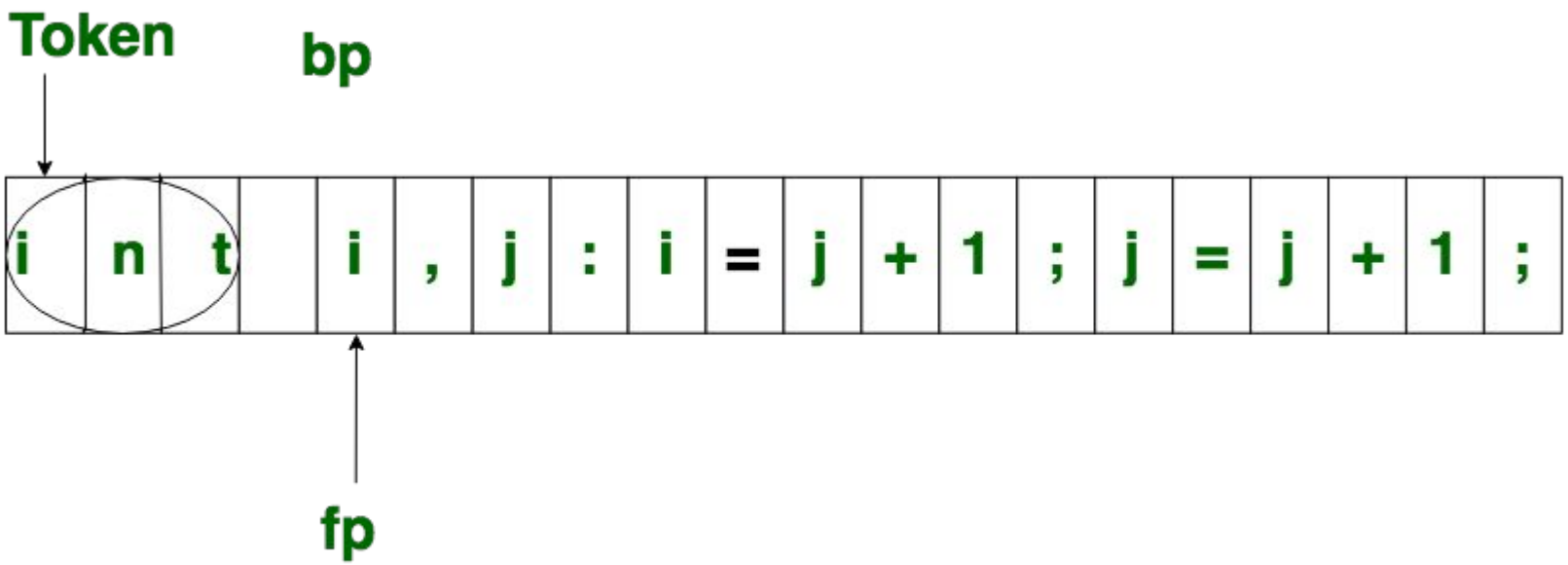


Initial Configuration

Input Buffering in Compiler Design

- The forward ptr moves ahead to search for end of lexeme.
- As soon as the blank space is encountered, it indicates end of lexeme.
- In above example as soon as ptr (fp) encounters a blank space the lexeme "int" is identified.
- The fp will be moved ahead at white space, when fp encounters white space, it ignore and moves ahead. then both the begin ptr(bp) and forward ptr(fp) are set at next token.
- The input character is thus read from secondary storage, but reading in this way from secondary storage is costly. hence buffering technique is used.
- A block of data is first read into a buffer, and then second by lexical analyzer. there are two methods used in this context: **One Buffer Scheme, and Two Buffer Scheme.**

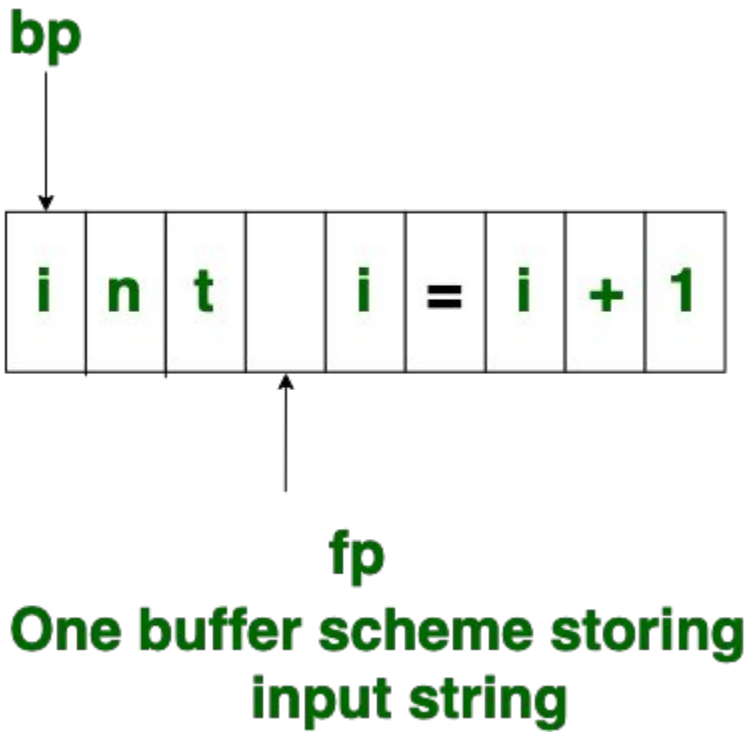
Input Buffering in Compiler Design



Input buffering

Input Buffering in Compiler Design

One Buffer Scheme: In this scheme, only one buffer is used to store the input string but the problem with this scheme is that if lexeme is very long then it crosses the buffer boundary, to scan rest of the lexeme the buffer has to be refilled, that makes overwriting the first of lexeme.



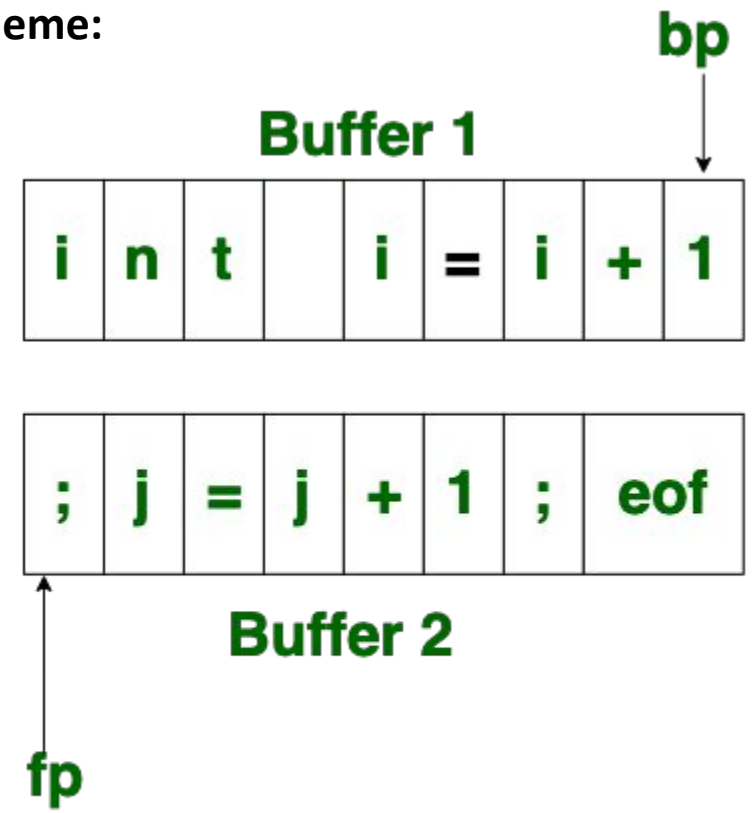
Input Buffering in Compiler Design

Two Buffer Scheme:

- To overcome the problem of one buffer scheme, in this method two buffers are used to store the input string.
- the first buffer and second buffer are scanned alternately. when end of current buffer is reached the other buffer is filled. the only problem with this method is that if length of the lexeme is longer than length of the buffer then scanning input cannot be scanned completely.
- Initially both the bp and fp are pointing to the first character of first buffer.Then the fp moves towards right in search of end of lexeme. as soon as blank character is recognized, the string between bp and fp is identified as corresponding token. to identify, the boundary of first buffer end of buffer character should be placed at the end first buffer.
- Similarly end of second buffer is also recognized by the end of buffer mark present at the end of second buffer. when fp encounters first **eof**, then one can recognize end of first buffer and hence filling up second buffer is started. in the same way when second **eof** is obtained then it indicates of second buffer.
- alternatively both the buffers can be filled up until end of the input program and stream of tokens is identified.
- This **eof** character introduced at the end is calling **Sentinel** which is used to identify the end of buffer.

Input Buffering in Compiler Design

Two Buffer Scheme:



Two buffer scheme storing input string

Input Buffering

Advantages:

- Input buffering can reduce the number of system calls required to read input from the source code, which can improve performance.
- Input buffering can simplify the design of the compiler by reducing the amount of code required to manage input.

Input Buffering

Disadvantages:

- If the size of the buffer is too large, it may consume too much memory, leading to slower performance or even crashes.
- If the buffer is not properly managed, it can lead to errors in the output of the compiler.
- Overall, the advantages of input buffering generally outweigh the disadvantages when used appropriately, as it can improve performance and simplify the compiler design.

Input Buffering

Disadvantages:

- If the size of the buffer is too large, it may consume too much memory, leading to slower performance or even crashes.
- If the buffer is not properly managed, it can lead to errors in the output of the compiler.
- Overall, the advantages of input buffering generally outweigh the disadvantages when used appropriately, as it can improve performance and simplify the compiler design.

SESSION 3

Finite Automata – Regular expression to Automata – Minimization of DFA